

■ 4.1.6 Object-Oriented Programming

When you write programs that involve many different functions, it is important to have some kind of general scheme for organizing your code. One traditional approach is to break your program into different “modules”, with each module containing all the functions that do a particular kind of thing.

Object-oriented programming is another approach to organizing code. The basic idea is to collect together not functions that do the same kinds of things, but rather functions that act on the same kinds of objects. Object-oriented programming is becoming very popular for organizing large software systems. In fact, the internal code of *Mathematica* itself was written using an object-oriented extension of the C programming language.

In the object-oriented approach, you introduce various types of object, and then specify “methods” for carrying out different operations on these objects. You can see how this works by thinking about writing a program to print out objects. The traditional approach would be to have one “print” function, which contains code for many different kinds of objects. The object-oriented approach is instead to associate with each type of object a function that gives the “method” for printing the object.

It is very easy to do object-oriented programming in *Mathematica*. The basic idea is to associate *Mathematica* transformation rules with the “objects” they act on, rather than with the functions they perform.

If you specify a rule for the pattern $f[x_]$, the rule will be associated with the symbol f . However, if you give a rule for $f[g[x_]]$, the rule can be associated either with f or with g .

This defines a rule for the “function” f . *In*[1] := $f[x_]$:= x^2

This rule is also associated with f . *In*[2] := $f[g[x_]]$:= $-g[-x]$

Here are the two rules you have given for f . *In*[3] := ? f
 f
 f /: $f[g[x_]]$:= $-g[-x]$
 f /: $f[x_]$:= x^2

The h /: tells *Mathematica* to associate this rule with h , rather than f . *In*[3] := h /: $f[h[x_]]$:= $p[x]$

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This shows the rule you have given for the “object” *h*.

```
In[4] := ?h
h
h/: f[h[x_]] := p[x]
```

You can set up different “types” of objects in *Mathematica* just by giving them different symbols as heads. Then you can associate rules or “methods” with these objects by including the appropriate */:* in the definitions you make.

```
ModularInteger/: Format[ModularInteger[i_, n_]] :=
    SequenceForm[i, " mod ", n]

ModularInteger/: Minus[ModularInteger[i_, n_]] := ModularInteger[Mod[-i, n], n]
```

Some rules for *ModularInteger* objects.

Object-oriented programming lets us organize definitions of mathematical objects in *Mathematica* in much the same way as the definitions might be organized in a mathematics textbook. You can have different “sections” of your code that give all the definitions associated with a particular kind of object. Using */:*, you can keep the definitions for each new kind of object separate.

```
ModularInteger/: ModularInteger[i_, n_] + ModularInteger[j_, n_] :=
    ModularInteger[Mod[i + j, n], n]
```

A rule for calculating sums of *ModularInteger* objects.

Operator Overloading

One important aspect of object-oriented programming in *Mathematica* is that it lets you “overload” operators like *+*. You can use the familiar *+* notation to represent any kind of addition. But exactly what the addition does can depend on the types of objects that are involved.

Associating definitions with types of objects rather than operations not only makes your code better organized, but can also make it much more efficient. The standard addition function *Plus* is used very often in *Mathematica*. If you gave additional definitions for *Plus* itself, then *Mathematica* would have to test these definitions every time it used the function *Plus*. The presence of your definitions would then slow down every single addition operation in *Mathematica*. On the other hand, if you associate your definitions with a particular type of object, *Mathematica* tests your definitions only when that particular object occurs. *Mathematica* would therefore usually not look at your definitions when it does additions, except in the specific cases where the arguments are of a particular type.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Mathematica always follows the general principle of looking at more specific definitions before more general ones. One consequence of this principle is that when *Mathematica* does an operation like addition, it looks at definitions that apply to the particular types of arguments used, before it looks at general definitions for addition with arbitrary types of arguments. As a result, your definitions for addition of a particular kind of object will always be tried before the standard built-in *Mathematica* addition function is used. You can effectively “override” the built-in addition function when a particular kind of object occurs.

The same mechanism allows you to insert your own definitions to specify how a particular kind of object should be printed. *Mathematica* tries your definitions before it uses its own standard built-in print functions.

In general, you can extend the way that *Mathematica*’s built-in functions work by adding definitions for how they should act on new kinds of objects that you introduce.

Sometimes you will need to introduce both new kinds of objects and new operations. You can give definitions for how your new operations act on various kinds of objects. Then you can give a more general definition for how your new operations act on other objects, for which you have not given more specific definitions. *Mathematica* will then first try to use the definitions you have given for specific types of objects; if it finds none that apply, then it will use the more general definition you have given for the operation.

Functions with Several Arguments

In object-oriented programming, each definition you make is associated with a particular kind of object. This approach works very well if each of your definitions involves only one kind of object. This is certainly true when you are dealing with functions like **Format** and **Minus** that take just one argument. However, things become more complicated when you need to deal with functions that take several arguments.

So long as all the arguments to a particular function are of the same type, things are still quite simple. The real problem comes when the arguments are of different types. What should *Mathematica* do when you ask to add a **ModularInteger** and a **PAdicInteger**? Should the result be a **ModularInteger** or a **PAdicInteger**? Sometimes you will just want to say that the objects you are trying to add together have incompatible types. But often, you will want to produce a definite result.

A common example that arises in many programming languages is the problem of combining different types of numbers. If you add an integer to a real number, you should get a real number as the result. If you add a rational number to an integer,

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

you should get a rational number. You can think of the different types of numbers as being arranged in the hierarchy: **Integer** $\rightarrow$ **Rational** $\rightarrow$ **Real**. Whenever you combine two numbers, they are both converted to the type that is higher on the hierarchy. If you add an integer to a real number, the integer is converted into a real number, and then the rule for adding two real numbers is used.

When several different kinds of objects appear in a particular definition, you can associate the definition with any of the kinds of objects. *Mathematica* will check all the objects in a particular expression to see if there are definitions associated with them.

Often you will want to associate your definitions with the least common object that appears. If you do this, *Mathematica* will test the definition as infrequently as possible.

By allowing you simply to associate transformation rules with any type of object, *Mathematica* gives you a flexible mechanism for dealing with expressions involving several different kinds of objects. One can imagine, however, developing a more structured approach to dealing with such expressions. At the simplest level, one might have a hierarchy of types of objects, and demand that when two objects occur together, they are always converted to the type that appears highest on the hierarchy. In practice, however, types of mathematical objects do not form any kind of strict hierarchy. In some respects, they form a network, or directed graph, and one could imagine always converting objects to the type of their “common ancestor” in the network. But even this much more elaborate mechanism does not adequately capture actual mathematical usage. Different forms of algebraic expressions, say factored and expanded out, can be thought of as different types. In most cases, the rules for when to convert between these types are much more complicated than you can represent by a simple fixed network of type conversions.